

Real-Time Chat Application — Project Interview Guide

Core 10 Project Breakdown & Architect Questions with Comprehensive Answers

Q1: What problem does your project solve?

Answer: Traditional web applications rely on HTTP polling or manual page refreshes, which cause high latency, excessive network header overhead, and poor user experiences for instant communication.

This project provides a low-latency, full-duplex, real-time messaging platform. It allows users to instantly exchange group room messages and private direct messages (DMs), receive dynamic typing indicators, view real-time online/offline statuses, and search through past message logs seamlessly without page reloads.

Q2: Why did you choose this project?

Answer: I chose this project to gain deep hands-on engineering experience with stateful, event-driven network protocols (WebSockets) and modern asynchronous Python frameworks (FastAPI).

While standard backend portfolio projects focus only on simple CRUD HTTP APIs, building a real-time chat application challenged me to tackle production system design topics such as long-lived socket lifecycle management, database connection pool optimization, JWT authentication over socket handshakes, and horizontal scaling patterns using pub/sub message brokers.

Q3: What was your role?

Answer: I served as the **Full-Stack / Lead Backend Engineer** for this application from end to end.

My primary responsibilities included: 1) Designing the relational database schema (User, Room, Message) and ORM relationships. 2) Developing the asynchronous FastAPI server and WebSocket ConnectionManager broker. 3) Implementing JWT authentication and REST endpoints for message history & search. 4) Creating a modern glassmorphic frontend UI with vanilla JavaScript and CSS. 5) Authoring automated pytest integration test suites and containerizing the app with Docker Compose.

Q4: What technologies did you use?

Answer:

- **Backend Framework:** Python 3.10+, FastAPI (ASGI Framework), WebSockets (websockets library).
- **Database & ORM:** SQLite (local dev & testing), MySQL 8.0 (production container), SQLAlchemy ORM.
- **Security & Data Validation:** PyJWT (JSON Web Tokens), Passlib with Bcrypt (password hashing), Pydantic (data schemas).
- **Frontend:** Vanilla HTML5, CSS3 (Glassmorphism design system, Flexbox/Grid), JavaScript (ES6+ native WebSocket API, Fetch API).
- **DevOps & Testing:** Docker, Docker Compose, Redis (caching/scaling layer), Pytest (automated testing).

Q5: Why did you choose this technology stack?

Answer: • **FastAPI:** Built on Starlette and Pydantic, offering high performance, native async/await event loops, and built-in WebSocket support.

• **SQLAlchemy & MySQL:** Provided robust ACID compliance, structured foreign key constraints, and seamless database abstraction between SQLite (dev) and MySQL (prod).

• **JWT Authentication:** Delivered a stateless, decentralized authentication mechanism without requiring server-side session lookup.

• **Vanilla JS & Glassmorphic CSS:** Allowed granular control over WebSocket events, DOM rendering, and custom micro-animations without external framework bundle overhead.

Q6: What was the architecture?

Answer: The application follows a decoupled, multi-layered architecture:

1. **Client Layer:** Browser client maintaining a persistent full-duplex WebSocket connection (`ws://`) alongside REST HTTP requests (`/api/`).

2. **REST API Layer:** FastAPI routers handling `/api/auth` (registration/login), `/api/users` (user profiles), and `/api/messages` (history logs and search).

3. **WebSocket Broker:** Central ConnectionManager keeping track of active user sockets, handling room subscriptions, broadcasting events, and delivering direct messages.

4. **Data Layer:** SQLAlchemy ORM managing persistence to SQLite/MySQL.

5. **Container Layer:** Docker Compose orchestrating the FastAPI server, MySQL database, and Redis cache.

Q7: What challenges did you face?

Answer: 1. **Database Connection Pool Exhaustion:** Standard FastAPI dependency injection kept database sessions open for the duration of long-lived WebSockets, quickly exhausting pool connections.

2. **JWT Authentication over WebSockets:** Standard browser WebSocket APIs do not support custom request headers (like Authorization: Bearer `<token>`) during socket handshakes.

3. **Event Spam & Stale States:** Rapid typing keypresses spammed network events, and abrupt tab closes left ghost 'online' user states.

Q8: How did you solve those challenges?

Answer: 1. **DB Session Decoupling:** Refactored ConnectionManager to use short-lived context managers (`with SessionLocal() as db:`) scoped strictly to individual read/write queries, returning connections to the pool immediately after commits.

2. **Query Token Handshake:** Passed JWT as a query parameter (`ws://domain/ws?token=JWT_TOKEN`), decoding signature and verifying user identity before accepting socket upgrades.

3. **Debouncing & Lifespan Cleanups:** Applied JavaScript debouncing (2s timeout) on typing inputs and added a FastAPI lifespan shutdown hook to automatically reset all users' `is_online = False` on server restarts.

Q9: What would you improve if you had more time?

Answer:

1. **Redis Pub/Sub Horizontal Scaling:** Implement Redis Pub/Sub out of the box so multiple backend instances behind a load balancer can share socket messages seamlessly.
2. **Alembic Database Migrations:** Integrate Alembic for tracking version-controlled database schema changes instead of relying on runtime `create_all()`.
3. **Media Attachments & Read Receipts:** Add S3 file upload endpoints for images/documents and track `read_at` timestamps for message read receipts.
4. **Frontend Framework:** Port the vanilla JS code to React or Next.js with TypeScript for cleaner component state management.

Q10: What did you personally implement?

Answer: I personally designed and developed the entire application end-to-end:

- Modeled relational database schemas (User, Room, Message) and Pydantic validation schemas.
- Built the asynchronous `ConnectionManager` class for routing WebSocket messages, typing status, and online user lists.
- Implemented JWT token generation, password hashing (`bcrypt`), registration, and login routes.
- Developed chat history pagination (`limit/offset`) and text search endpoints.
- Designed and coded the modern glassmorphic dark-mode UI with dynamic responsive layouts.
- Containerized the application using Docker & Docker Compose and wrote automated pytest test suites.